

Practical No. 1

Aim: Write programs to parse text files, CSV, HTML, XML and JSON documents and extract.

1. Text File

```
# Create a sample text file

text_data = """Name: Rahul
Age: 21
City: Mumbai
Course: Data Science
Name: Priya
Age: 22
City: Pune
Course: Data Science
"""

with open("students.txt", "w") as file:
    file.write(text_data)

print("Text file created successfully.")

# Read the text file

with open("students.txt", "r") as file:
    data = file.read()

print(data)
# Extract information from the text file

with open("students.txt", "r") as file:
    lines = file.readlines()

names = []
ages = []
cities = []
courses = []

for line in lines:
    line = line.strip()

    if line.startswith("Name:"):
        names.append(line.split(":")[1].strip())
```

```

elif line.startswith("Age:"):
    ages.append(line.split(":")[1].strip())
elif line.startswith("City:"):
    cities.append(line.split(":")[1].strip())

    elif line.startswith("Course:"):
        courses.append(line.split(":")[1].strip())

print("Names:", names)
print("Ages:", ages)
print("Cities:", cities)
print("Courses:", courses)
# Check for missing values

print("Missing Names:", sum(1 for x in names if x == ""))
print("Missing Ages:", sum(1 for x in ages if x == ""))
print("Missing Cities:", sum(1 for x in cities if x == ""))
print("Missing Courses:", sum(1 for x in courses if x == ""))
# Convert ages into integers
ages = [int(age) for age in ages]

# Check for age anomalies
anomalies = [age for age in ages if age < 0 or age > 100]
print("Age anomalies:", anomalies)

if len(anomalies) == 0:
    print("No age anomalies found.")
else:
    print("Anomalies found:", anomalies)

```

Output:

```

... Name: Rahul
    Age: 21
    City: Mumbai
    Course: Data Science
    Name: Priya
    Age: 22
    City: Pune
    Course: Data Science

```

```

• Names: ['Rahul', 'Priya']
  Ages: ['21', '22']
  Cities: ['Mumbai', 'Pune']
  Courses: ['Data Science', 'Data Science']

```

```

Missing Names: 0
Missing Ages: 0
Missing Cities: 0
Missing Courses: 0

```

```

Age anomalies: []
No age anomalies found.

```

2. CSV File

```
import pandas as pd
# Create sample data
data = {
    "Name": ["Rahul", "Priya", "Amit", "Sneha", "Rohan"],
    "Age": [21, 22, 20, None, 150],
    "City": ["Mumbai", "Pune", "Nashik", "Mumbai", None],
    "Marks": [85, 90, 78, 88, -10]
}

df = pd.DataFrame(data)

# Save as CSV
df.to_csv("students.csv", index=False)

print("CSV file created successfully.")

# Read CSV file
df = pd.read_csv("students.csv")

print(df)

# Check missing values
print("Missing values:")
print(df.isnull().sum())

# Check duplicate records
print("Number of duplicate rows:", df.duplicated().sum())

# Check age anomalies
age_anomalies = df[(df["Age"] < 0) | (df["Age"] > 100)]

print("Age anomalies:")
print(age_anomalies)

# Check marks anomalies
marks_anomalies = df[(df["Marks"] < 0) | (df["Marks"] > 100)]

print("Marks anomalies:")
print(marks_anomalies)
# Display basic information
print(df.info())
# Statistical summary
print(df.describe())
```

Output:

```
***      Name    Age    City    Marks
0    Rahul    21.0    Mumbai    85
1    Priya    22.0     Pune    90
2     Amit    20.0    Nashik    78
3    Sneha     NaN    Mumbai    88
4     Rohan   150.0      NaN   -10
```

Missing values:

```
Name      0
Age       1
City      1
Marks     0
dtype: int64
```

Number of duplicate rows: 0

Age anomalies:

```
      Name    Age    City    Marks
4    Rohan   150.0     NaN   -10
```

Marks anomalies:

```
      Name    Age    City    Marks
4    Rohan   150.0     NaN   -10
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  ---
0    Name    5 non-null      object
1    Age      4 non-null      float64
2    City     4 non-null      object
3    Marks    5 non-null      int64
dtypes: float64(1), int64(1), object(2)
memory usage: 292.0+ bytes
None
```

```
***      Age      Marks
count    4.000000    5.000000
mean     53.250000    66.200000
std       64.505168    42.839234
min       20.000000   -10.000000
25%       20.750000    78.000000
50%       21.500000    85.000000
75%       54.000000    88.000000
max      150.000000    90.000000
```

3. HTML File

```
html_data = """
<html>
<body>

<h2>Student Details</h2>

<table border="1">
<tr>
<th>Name</th>
<th>Age</th>
<th>City</th>
<th>Marks</th>
</tr>

<tr>
<td>Rahul</td>
<td>21</td>
<td>Mumbai</td>
<td>85</td>
</tr>

<tr>
<td>Priya</td>
<td>22</td>
<td>Pune</td>
<td>90</td>
</tr>

<tr>
<td>Amit</td>
<td>20</td>
<td>Nashik</td>
<td>78</td>
</tr>

</table>

</body>
</html>
"""

with open("students.html", "w") as file:
    file.write(html_data)
print("HTML file created successfully.")
```

```

# Read tables from HTML

tables = pd.read_html("students.html")

html_df = tables[0]

print(html_df)

print("Missing values:")
print(html_df.isnull().sum())

# Check age anomalies

print("Age anomalies:")
print(html_df[(html_df["Age"] < 0) | (html_df["Age"] > 100)])

# Check marks anomalies

print("Marks anomalies:")
print(html_df[(html_df["Marks"] < 0) | (html_df["Marks"] > 100)])

```

Output:

```

...
   Name  Age  City  Marks
0  Rahul   21  Mumbai    85
1  Priya   22   Pune    90
2   Amit   20  Nashik    78

```

```

... Missing values:
Name      0
Age       0
City      0
Marks     0
dtype: int64

```

```

Age anomalies:
Empty DataFrame
Columns: [Name, Age, City, Marks]
Index: []

```

```

... Marks anomalies:
Empty DataFrame
Columns: [Name, Age, City, Marks]
Index: []

```

4. XML File

```
xml_data = """<?xml version="1.0"?>
<students>
    <student>
        <name>Rahul</name>
        <age>21</age>
        <city>Mumbai</city>
        <marks>85</marks>
    </student>

    <student>
        <name>Priya</name>
        <age>22</age>
        <city>Pune</city>
        <marks>90</marks>
    </student>

    <student>
        <name>Amit</name>
        <age>20</age>
        <city>Nashik</city>
        <marks>78</marks>
    </student>

</students>
"""

with open("students.xml", "w") as file:
    file.write(xml_data)
print("XML file created successfully.")
import xml.etree.ElementTree as ET
tree = ET.parse("students.xml")
root = tree.getroot()
print("Root element:", root.tag)
names = []
ages = []
cities = []
marks = []
for student in root.findall("student"):

    names.append(student.find("name").text)
    ages.append(int(student.find("age").text))
    cities.append(student.find("city").text)
    marks.append(int(student.find("marks").text))
```

```

xml_df = pd.DataFrame({
    "Name": names,
    "Age": ages,
    "City": cities,
    "Marks": marks
})

print(xml_df)

print("Missing values:")
print(xml_df.isnull().sum())
print("Age anomalies:")
print(xml_df[(xml_df["Age"] < 0) | (xml_df["Age"] > 100)])

print("Marks anomalies:")
print(xml_df[(xml_df["Marks"] < 0) | (xml_df["Marks"] > 100)])

```

Output:

```
... Root element: students
```

```
... Names: ['Rahul', 'Priya', 'Amit']
Ages: [21, 22, 20]
Cities: ['Mumbai', 'Pune', 'Nashik']
Marks: [85, 90, 78]
```

```
...
  Name  Age  City  Marks
0  Rahul  21  Mumbai   85
1  Priya  22   Pune   90
2   Amit  20  Nashik   78
```

```
... Missing values:
Name      0
Age       0
City      0
Marks     0
dtype: int64
```

```
Age anomalies:
Empty DataFrame
Columns: [Name, Age, City, Marks]
Index: []
Marks anomalies:
Empty DataFrame
Columns: [Name, Age, City, Marks]
Index: []
```

5. JSON File

```
import json

students = [
    {
        "Name": "Rahul",
        "Age": 21,
        "City": "Mumbai",
        "Marks": 85
    },
    {
        "Name": "Priya",
        "Age": 22,
        "City": "Pune",
        "Marks": 90
    },
    {
        "Name": "Amit",
        "Age": 20,
        "City": "Nashik",
        "Marks": 78
    }
]

with open("students.json", "w") as file:
    json.dump(students, file, indent=4)

print("JSON file created successfully.")

with open("students.json", "r") as file:
    data = json.load(file)
print(data)
for student in data:
    print("Name:", student["Name"])
    print("Age:", student["Age"])
    print("City:", student["City"])
    print("Marks:", student["Marks"])
    print()
json_df = pd.DataFrame(data)

print(json_df)
```

```

print("Missing values:")
print(json_df.isnull().sum())

print("Age anomalies:")
print(json_df[(json_df["Age"] < 0) | (json_df["Age"] > 100)])

print("Marks anomalies:")
print(json_df[(json_df["Marks"] < 0) | (json_df["Marks"] > 100)])

```

Output:

```
[{'Name': 'Rahul', 'Age': 21, 'City': 'Mumbai', 'Marks': 85}, {'Name': 'Priya', 'Age': 22, 'City': 'Pune', 'Marks': 90}, {'Name': 'Amit', 'Age': 20, 'City': 'Nashik', 'Marks': 78}]
```

```

.. Name: Rahul
   Age: 21
   City: Mumbai
   Marks: 85

```

```

Name: Priya
Age: 22
City: Pune
Marks: 90

```

```

Name: Amit
Age: 20
City: Nashik
Marks: 78

```

	Name	Age	City	Marks
0	Rahul	21	Mumbai	85
1	Priya	22	Pune	90
2	Amit	20	Nashik	78

```

.. Missing values:
   Name      0
   Age       0
   City       0
   Marks      0
   dtype: int64

```

```

Age anomalies:
Empty DataFrame
Columns: [Name, Age, City, Marks]
Index: []
Marks anomalies:
Empty DataFrame
Columns: [Name, Age, City, Marks]
Index: []

```

Aim: Write programs for reading and writing binary files.

```
import pickle

# Sample student data
student = {
    "Name": "Rahul",
    "Age": 21,
    "Course": "Data Science",
    "Marks": 85
}

print("Original Data:")
print(student)

# Writing data to a binary file
with open("student.dat", "wb") as file:
    pickle.dump(student, file)
print("Data successfully written to binary file.")

# Reading data from binary file

with open("student.dat", "rb") as file:
    data = pickle.load(file)

print("Data read from binary file:")
print(data)

# Data to be written
student = {
    "Name": "Rahul",
    "Age": 21,
    "Course": "Data Science",
    "Marks": 85
}

# Writing to binary file
with open("student.dat", "wb") as file:
    pickle.dump(student, file)
print("Data written successfully.")

# Reading from binary file
with open("student.dat", "rb") as file:
    data = pickle.load(file)
print("Data read successfully.")
print("Student Data:")
print(data)
```

Output:

```
... Original Data:  
{'Name': 'Rahul', 'Age': 21, 'Course': 'Data Science', 'Marks': 85}
```

Data successfully written to binary file.

```
Data read from binary file:  
{'Name': 'Rahul', 'Age': 21, 'Course': 'Data Science', 'Marks': 85}
```

```
Data written successfully.  
Data read successfully.  
Student Data:  
{'Name': 'Rahul', 'Age': 21, 'Course': 'Data Science', 'Marks': 85}
```

Aim: Write programs for searching, splitting, and replacing strings based on pattern matching using regular expressions.

```
import re

text = "Contact Rahul at rahul@gmail.com for more information."

pattern = r"\w+@\w+\.\w+"

result = re.search(pattern, text)

if result:
    print("Email found:", result.group())
else:
    print("Email not found.")

text = "Rahul, Priya; Amit, Sneha; Rohan"

result = re.split(r"[,;]", text)

print("After splitting:")
print(result)

text = "Rahul scored 85 marks and Priya scored 90 marks."

result = re.sub(r"\d+", "XXX", text)

print("Original text:")
print(text)

print("\nAfter replacement:")
print(result)

# Sample text
text = "Rahul scored 85 marks. Contact him at rahul@gmail.com; Priya scored 90 marks."
# 1. Searching
pattern = r"\w+@\w+\.\w+"

result = re.search(pattern, text)
if result:
    print("Email found:", result.group())
else:
    print("Email not found.")
```

```
# 2. Splitting
split_result = re.split(r"[;]", text)

print("\nAfter splitting:")
print(split_result)

# 3. Replacing
replace_result = re.sub(r"\d+", "XXX", text)

print("\nAfter replacing numbers:")
print(replace_result)
```

Output:

Email found: rahul@gmail.com

After splitting:
['Rahul', ' Priya', ' Amit', ' Sneha', ' Rohan']

Original text:
Rahul scored 85 marks and Priya scored 90 marks.

After replacement:
Rahul scored XXX marks and Priya scored XXX marks.

Email found: rahul@gmail.com

After splitting:
['Rahul scored 85 marks. Contact him at rahul@gmail.com', ' Priya scored 90 marks.']

After replacing numbers:
Rahul scored XXX marks. Contact him at rahul@gmail.com; Priya scored XXX marks.

Aim: 4. Design a relational database for a small application and populate the database. Using SQL do the CRUD (Create, Read, Update and Delete) operations.

```
import sqlite3

# Connect to database
connection = sqlite3.connect("college.db")
cursor = connection.cursor()

# CREATE TABLE
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER,
    course TEXT,
    marks INTEGER
)
""")

connection.commit()

print("Table created successfully.")

# INSERT RECORDS
students = [
    (1, "Rahul", 21, "Data Science", 85),
    (2, "Priya", 22, "Computer Science", 90),
    (3, "Amit", 20, "Data Science", 78),
    (4, "Sneha", 21, "Artificial Intelligence", 88)
]
cursor.executemany("""
INSERT OR IGNORE INTO students
(id, name, age, course, marks)
VALUES (?, ?, ?, ?, ?)
""", students)

connection.commit()

print("Records inserted successfully.")

# CREATE - Insert a new record
cursor.execute("""
INSERT OR IGNORE INTO students
(id, name, age, course, marks)
VALUES (5, 'Rohan', 22, 'Data Science', 82)
""")
```

```

"""
connection.commit()

print("\nCREATE operation completed.")

# READ
print("\nREAD operation:")
cursor.execute("SELECT * FROM students")

for record in cursor.fetchall():
    print(record)

# UPDATE
cursor.execute("""
UPDATE students
SET marks = 92
WHERE id = 1
""")

connection.commit()

print("\nUPDATE operation completed.")

# Display updated record
cursor.execute("SELECT * FROM students WHERE id = 1")
print("Updated record:", cursor.fetchone())

# DELETE
cursor.execute("""
DELETE FROM students
WHERE id = 5
""")

connection.commit()

print("\nDELETE operation completed.")

# Display final records
print("\nFinal records:")
cursor.execute("SELECT * FROM students")

for record in cursor.fetchall():
    print(record)

```

```
print("\nFinal records:")

cursor.execute("SELECT * FROM students")

for record in cursor.fetchall():
    print(record)

# Close connection
connection.close()

print("\nDatabase connection closed.")
```

Output:

```
Table created successfully.
Records inserted successfully.

CREATE operation completed.

READ operation:
(1, 'Rahul', 21, 'Data Science', 92)
(2, 'Priya', 22, 'Computer Science', 90)
(3, 'Amit', 20, 'Data Science', 78)
(4, 'Sneha', 21, 'Artificial Intelligence', 88)
(5, 'Rohan', 22, 'Data Science', 82)

UPDATE operation completed.
Updated record: (1, 'Rahul', 21, 'Data Science', 92)

DELETE operation completed.

Final records:
(1, 'Rahul', 21, 'Data Science', 92)
(2, 'Priya', 22, 'Computer Science', 90)
(3, 'Amit', 20, 'Data Science', 78)
(4, 'Sneha', 21, 'Artificial Intelligence', 88)

Database connection closed.
```

Practical No. 2

Aim: Perform the Extraction Transformation and Loading (ETL) process to construct the database in the Sqlserver / Power BI.

Step 1: Data Extraction – Importing Northwind Dataset into Power BI

D:\Northwind							
Content	Name	Extension	Date accessed	Date modified	Date created	Attributes	Folder Path
Binary	categories.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Record	D:\Northwind\
Binary	customers.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Record	D:\Northwind\
Binary	employees.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Record	D:\Northwind\
Binary	orders.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Record	D:\Northwind\
Binary	order_details.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Record	D:\Northwind\
Binary	products.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Record	D:\Northwind\
Binary	shippers.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Record	D:\Northwind\

Step 2: Data loaded into Power Query Editor

= Folder.Files("D:\Northwind")							
Content	Name	Extension	Date accessed	Date modified	Date created	Attributes	Folder Path
1 Binary	categories.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Reco	
2 Binary	customers.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Reco	
3 Binary	employees.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Reco	
4 Binary	orders.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Reco	
5 Binary	order_details.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Reco	
6 Binary	products.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Reco	
7 Binary	shippers.csv	.csv	15-08-2026 15:14:06	15-08-2026 15:14:06	23-06-2023 18:38:28	Reco	

Step 3: Required Northwind tables imported into Power Query

= Table.TransformColumnTypes(#"Promoted Headers",{{"orderID", type text}, {"productID", type text}, {"unitPrice", type text}}})			
1 ² orderID	1 ² productID	1.2 unitPrice	
1	10248	11	
2	10248	42	
3	10248	72	
4	10249	14	
5	10249	51	
6	10250	41	
7	10250	51	

Step 4: Removing unnecessary columns from the Products table

Queries [3] ✕ ✓ fx `= Table.SelectColumns("#Changed Type",{"productName", "productID", "quantityPerUnit", "unitPrice"})`

products	productID	productName	quantityPerUnit	unitPrice
1	1	Chai	10 boxes x 20 bags	18
2	2	Chang	24 - 12 oz bottles	19
3	3	Aniseed Syrup	12 - 550 ml bottles	10
4	4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	22
5	5	Chef Anton's Gumbo Mix	36 boxes	21.35
6	6	Grandma's Boysenberry Spread	12 - 8 oz jars	25
7	7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	30
8	8	Northwoods Cranberry Sauce	12 - 12 oz jars	40
9	9	Mishi Kobe Niku	18 - 500 g pkgs.	97
10	10	Ikura	12 - 200 ml jars	31
11	11	Queso Cabrales	1 kg pkg.	21

Step 5: Merging Orders with Order Details using OrderID

Queries [3] ✕ ✓ fx `= Table.NestedJoin("#Changed Type", {"orderID"}, order_details, {"orderID"}, "order_details", JoinKind.LeftOuter)`

orderDate	requiredDate	shippedDate	shipperID	freight	order_details
1	5	04-07-2013	01-08-2013	16-07-2013	3 32.38 Table
2	6	05-07-2013	16-08-2013	10-07-2013	1 11.61 Table
3	4	08-07-2013	05-08-2013	12-07-2013	2 65.83 Table
4	3	08-07-2013	05-08-2013	15-07-2013	1 41.34 Table
5	4	09-07-2013	06-08-2013	11-07-2013	2 51.3 Table
6	3	10-07-2013	24-07-2013	16-07-2013	2 58.17 Table
7	5	11-07-2013	08-08-2013	23-07-2013	2 22.98 Table
8	9	12-07-2013	09-08-2013	15-07-2013	3 148.33 Table
9	3	15-07-2013	12-08-2013	17-07-2013	2 13.97 Table
10	4	16-07-2013	13-08-2013	22-07-2013	3 81.91 Table
11	1	17-07-2013	14-08-2013	23-07-2013	1 140.51 Table
12	4	18-07-2013	15-08-2013	25-07-2013	3 3.25 Table
13	4	19-07-2013	16-08-2013	29-07-2013	1 55.09 Table
14	4	19-07-2013	16-08-2013	30-07-2013	2 3.05 Table
15	8	22-07-2013	19-08-2013	25-07-2013	3 48.29 Table
16	9	23-07-2013	20-08-2013	31-07-2013	3 146.06 Table
17	6	24-07-2013	21-08-2013	23-08-2013	3 3.67 Table
18	2	25-07-2013	22-08-2013	12-08-2013	1 55.28 Table
19	3	26-07-2013	06-09-2013	31-07-2013	3 25.73 Table
20	4	29-07-2013	26-08-2013	06-08-2013	1 208.58 Table
21	8	30-07-2013	27-08-2013	02-08-2013	3 66.29 Table
22	5	31-07-2013	14-08-2013	09-08-2013	1 4.56 Table
23	1	01-08-2013	29-08-2013	02-08-2013	1 136.54 Table
24	6	01-08-2013	29-08-2013	30-08-2013	2 4.54 Table
25	6	02-08-2013	30-08-2013	06-08-2013	2 98.03 Table
26	3	05-08-2013	02-09-2013	12-08-2013	3 76.07 Table
27					

9 COLUMNS, 830 ROWS Column profiling based on top 1000 rows

Step 6: Expanding Order Details and selecting ProductID, UnitPrice and Quantity

Queries [3] ✕ ✓ fx `= Table.ExpandTableColumn("#Merged Queries", "order_details", {"productID", "unitPrice", "quantity"}, {"productID", "unitPrice", "quantity"})`

shippedDate	shipperID	freight	productID	unitPrice	quantity
1	3	16-07-2013	3	32.38	14
2	3	16-07-2013	3	32.38	9.8
3	3	16-07-2013	3	32.38	34.8
4	3	10-07-2013	1	11.61	18.6
5	3	10-07-2013	1	11.61	42.4
6	3	12-07-2013	2	65.83	7.7
7	3	12-07-2013	2	65.83	42.4
8	3	12-07-2013	2	65.83	16.8
9	3	15-07-2013	1	41.34	16.8
10	3	15-07-2013	1	41.34	15.6
11	3	15-07-2013	1	41.34	16.8
12	3	11-07-2013	2	51.3	64.8
13	3	11-07-2013	2	51.3	2
14	3	11-07-2013	2	51.3	27.2
15	3	16-07-2013	2	58.17	10
16	3	16-07-2013	2	58.17	14.4
17	3	16-07-2013	2	58.17	16
18	3	23-07-2013	2	22.98	3.6
19	3	23-07-2013	2	22.98	19.2
20	3	23-07-2013	2	22.98	8

Step 7: Creating LineTotal using UnitPrice × Quantity

tal", each [unitPrice] * [quantity])

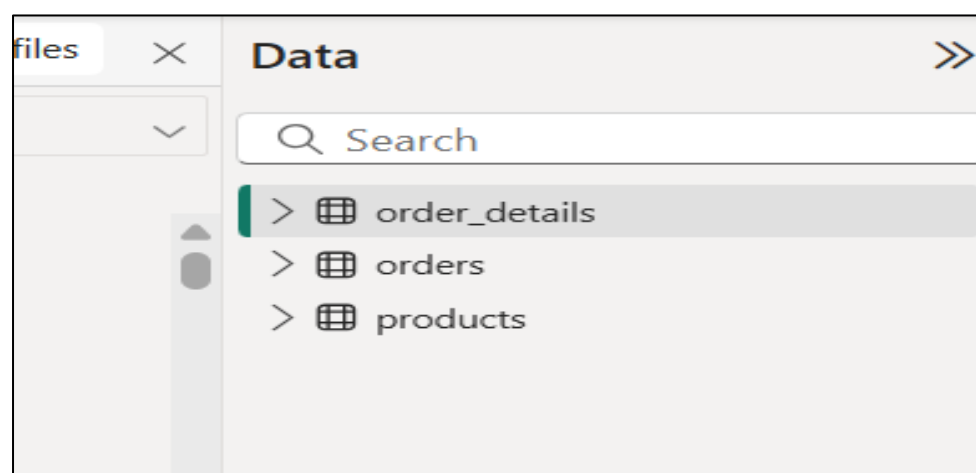
	1.2 unitPrice	1.2 quantity	ABC 123 LineTotal
11	14	12	168
42	9.8	10	98
72	34.8	5	174
14	18.6	9	167.4
51	42.4	40	1696
41	7.7	10	77
51	42.4	35	1484
65	16.8	15	252
22	16.8	6	100.8

Step 8: Reordering the transformed columns

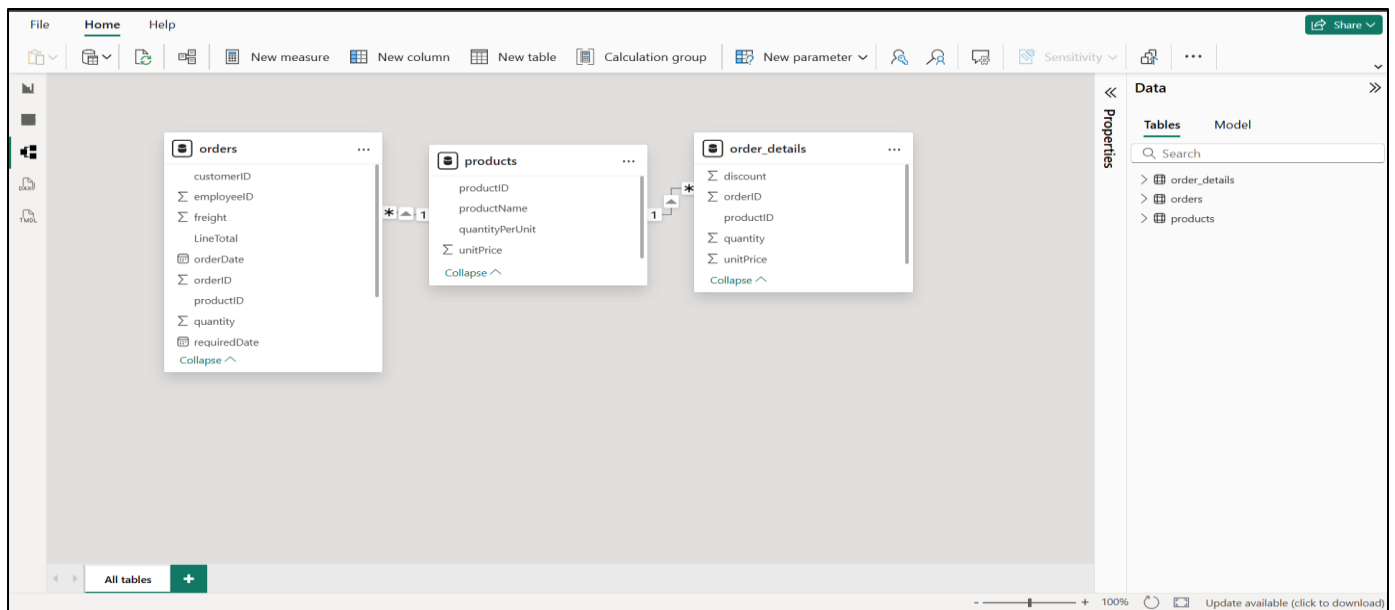
Formula Bar: = Table.ReorderColumns("#Added Custom",{ "orderId", "customerID", "employeeID", "orderDate", "requiredDate", "

	1.2 shipperID	1.2 freight	ABC 123 LineTotal	1.2 productID	1.2 unitPrice
1	2013	3	32.38	168	11
2	2013	3	32.38	98	42
3	2013	3	32.38	174	72
4	2013	1	11.61	167.4	14
5	2013	1	11.61	1696	51
6	2013	2	65.83	77	41
7	2013	2	65.83	1484	51
8	2013	2	65.83	252	65

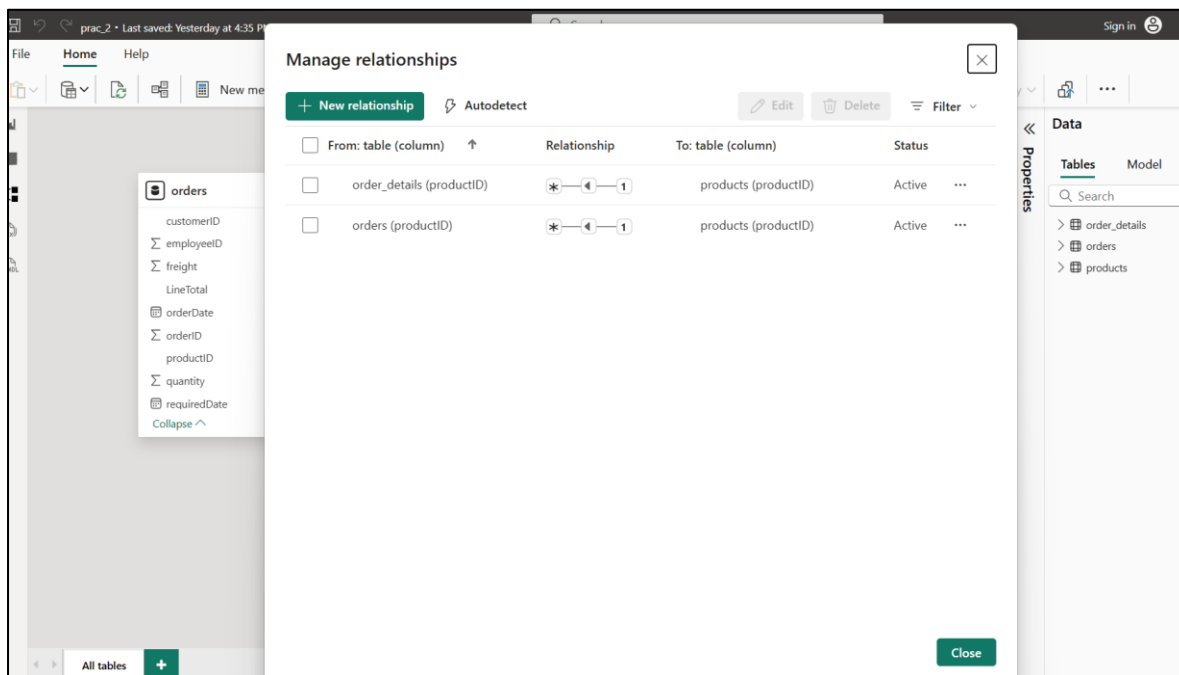
Step 9: Loading the transformed data into Power BI



Step 10: Creating relationships between tables using ProductID



Step 11: Verifying the existing relationships between Products, Orders and Order Details



Practical No. 3

Aim: To perform basic MongoDB document operations by inserting single and multiple documents into collections and retrieving the inserted documents using MongoDB commands.

Step 1 — Insert Single Document

```
test> use practical3
switched to db practical3
practical3> db.items.insertOne({ name: "laptop", price: 999 });
{
  acknowledged: true,
  insertedId: ObjectId('6a86676d4b503bee472af196')
}
```

Step 2 — View the inserted document

```
practical3> db.items.find();
[
  {
    _id: ObjectId('6a86676d4b503bee472af196'),
    name: 'laptop',
    price: 999
  }
]
practical3> |
```

Step 3 — Insert Multiple Documents

```
practical3> db.products.insertMany([
|   { name: "phone", price: 500, stock: 10 },
|   { name: "tablet", price: 300, stock: 5 },
|   { name: "watch", price: 150, stock: 0 }
| ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a8669d64b503bee472af197'),
    '1': ObjectId('6a8669d64b503bee472af198'),
    '2': ObjectId('6a8669d64b503bee472af199')
  }
}
practical3> |
```

Step 4 — View Products

```
practical3> db.products.find().pretty();
[
  {
    _id: ObjectId('6a8669d64b503bee472af197'),
    name: 'phone',
    price: 500,
    stock: 10
  },
  {
    _id: ObjectId('6a8669d64b503bee472af198'),
    name: 'tablet',
    price: 300,
    stock: 5
  },
  {
    _id: ObjectId('6a8669d64b503bee472af199'),
    name: 'watch',
    price: 150,
    stock: 0
  }
]
practical3> |
```

Practical No. 4

Aim: To perform noise elimination, feature selection, and exploratory data analysis (EDA) on a dataset using Pandas, NumPy, Matplotlib, and Scikit-learn.

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_selection import VarianceThreshold

# Create Sample Dataset

data = {
    'Age': [20, 21, 22, 23, 24, 100],          # 100 is an outlier
    'Marks': [75, 80, 85, 90, 95, 20],
    'Attendance': [85, 88, 90, 92, 94, 50],
    'Constant': [1, 1, 1, 1, 1, 1]           # Constant feature
}

df = pd.DataFrame(data)
print("Original Dataset:")
print(df)

# 1. Noise Elimination - Remove Outlier
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Remove Age values outside the acceptable range
df_clean = df[(df['Age'] >= lower) & (df['Age'] <= upper)]

print("\nDataset after Noise Elimination:")
print(df_clean)

# 2. Feature Selection

# Remove constant features using VarianceThreshold
selector = VarianceThreshold(threshold=0.0)
selected = selector.fit_transform(df_clean)

selected_columns = df_clean.columns[selected.get_support()]
```

```

selected_columns = df_clean.columns[selector.get_support()]
print("\nSelected Features:")
print(selected_columns)
# 3. Exploratory Data Analysis (EDA)

# Display basic information about the dataset
print("\nDataset Information:")
df_clean.info()

# Display statistical summary
print("\nStatistical Summary:")
print(df_clean.describe())

# Display correlation matrix
print("\nCorrelation Matrix:")
print(df_clean.corr())

# Histogram

df_clean.hist(figsize=(8, 6))
plt.suptitle("Histogram of Features")
plt.show()

# Boxplot
df_clean.boxplot(figsize=(8, 5))
plt.title("Boxplot of Dataset")
plt.show()

```

Output:

```

... Original Dataset:
   Age  Marks  Attendance  Constant
0   20    75         85         1
1   21    80         88         1
2   22    85         90         1
3   23    90         92         1
4   24    95         94         1
5  100    20         50         1

Dataset after Noise Elimination:
   Age  Marks  Attendance  Constant
0   20    75         85         1
1   21    80         88         1
2   22    85         90         1
3   23    90         92         1
4   24    95         94         1

Selected Features:
Index(['Age', 'Marks', 'Attendance'], dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
Index: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Age          5 non-null      int64
1   Marks        5 non-null      int64
2   Attendance   5 non-null      int64
3   Constant     5 non-null      int64
dtypes: int64(4)
memory usage: 200.0 bytes

```

```

Statistical Summary:
      Age      Marks  Attendance  Constant
count  5.000000  5.000000    5.00000    5.0
mean   22.000000  85.000000   89.80000    1.0
std     1.581139   7.905694    3.49285    0.0
min    20.000000  75.000000   85.00000    1.0
25%    21.000000  80.000000   88.00000    1.0
50%    22.000000  85.000000   90.00000    1.0
75%    23.000000  90.000000   92.00000    1.0
max    24.000000  95.000000   94.00000    1.0

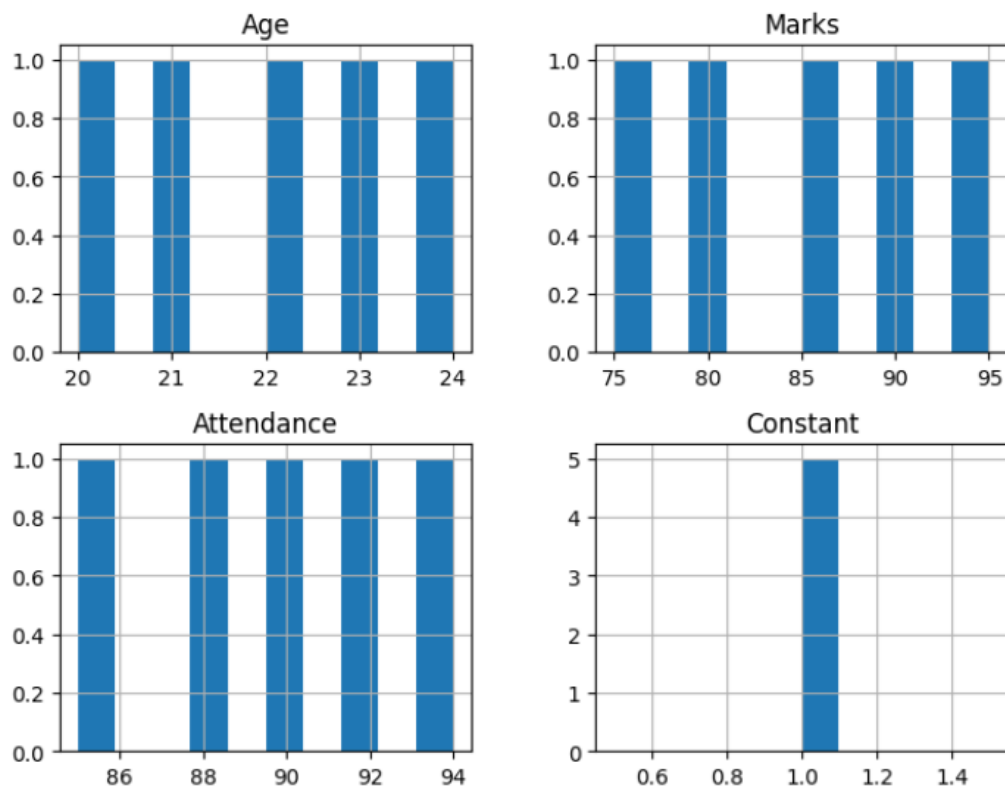
```

```

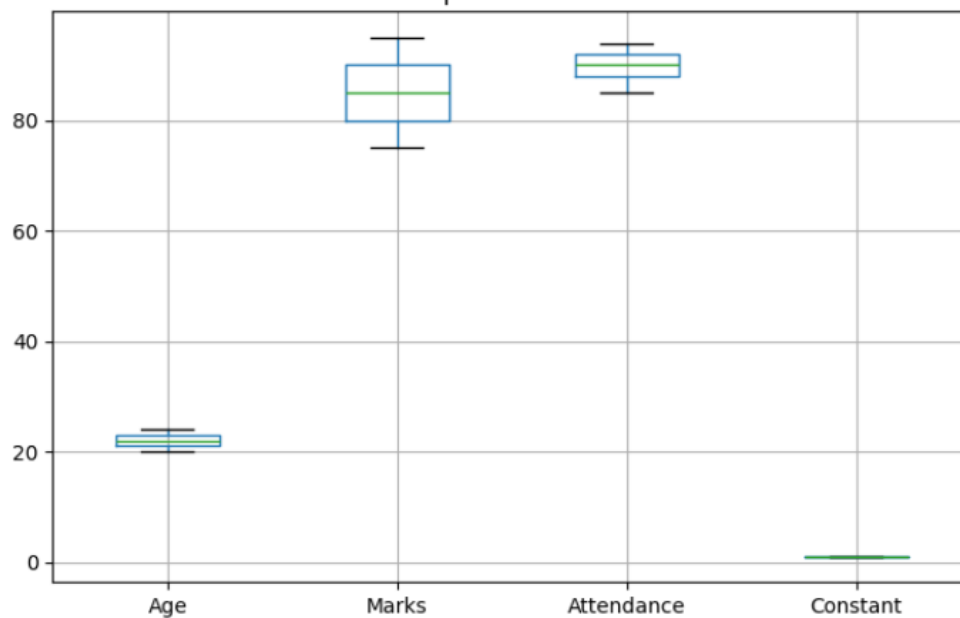
Correlation Matrix:
      Age      Marks  Attendance  Constant
Age      1.000000  1.000000    0.995893    NaN
Marks      1.000000  1.000000    0.995893    NaN
Attendance  0.995893  0.995893    1.000000    NaN
Constant   NaN      NaN      NaN      NaN

```

Histogram of Features



Boxplot of Dataset



Practical No. 5

Aim: To extract data from a public REST API and flat CSV file, perform data cleaning and transformation, merge the datasets, and store the processed data for further use.

```
# Import required libraries
import pandas as pd
import requests

# Function to Extract Data from REST API
def extract_api_data(url):
    """Fetch data from a REST API and return a DataFrame."""
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()

        data = response.json()

        # Convert JSON data into a tabular format
        return pd.json_normalize(data)

    except requests.exceptions.RequestException as e:
        print(f"API Error: {e}")
        return pd.DataFrame()

# Function to Extract Data from CSV File
def extract_csv_data(file_path):
    """Read data from a CSV file."""
    try:
        return pd.read_csv(file_path)

    except FileNotFoundError as e:
        print(f"File Error: {e}")
        return pd.DataFrame()

# Step 1: Extract Data from REST API
api_url = "https://jsonplaceholder.typicode.com/users"

api_df = extract_api_data(api_url)

# Select required columns
api_df = api_df[["id", "name", "email", "company.name"]]

# Rename column for simplicity
api_df.rename(columns={"company.name": "company"}, inplace=True)

print("--- API Data ---")
print(api_df.head())
```

```

# Step 2: Create and Read Flat CSV File
csv_file = "locations.csv"

# Create sample location data
location_data = pd.DataFrame({
    "id": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "city": [
        "New York",
        "London",
        "Paris",
        "Tokyo",
        "Berlin",
        "Delhi",
        "Sydney",
        "Moscow",
        "Cairo",
        "Beijing"
    ],
    "country": [
        "USA",
        "UK",
        "France",
        "Japan",
        "Germany",
        "India",
        "Australia",
        "Russia",
        "Egypt",
        "China"
    ]
})

# Save the sample data as a CSV file
location_data.to_csv(csv_file, index=False)

# Read the CSV file
csv_df = extract_csv_data(csv_file)
print("\n--- CSV Data ---")
print(csv_df)

# Step 3: Merge API and CSV Data
if not api_df.empty and not csv_df.empty:
    # Merge both datasets using the common 'id' column
    merged_df = pd.merge(
        api_df,
        csv_df,

```

```

on="id",
    how="inner"
)

print("\n--- Merged ETL Pipeline Data ---")
print(merged_df)

# Step 4: Save Cleaned Data

merged_df.to_csv(
    "cleaned_warehouse_profiles.csv",
    index=False
)
print("\nData successfully saved to 'cleaned_warehouse_profiles.csv'")

```

Output:

--- API Data ---

	id	name	email	company
0	1	Leanne Graham	Sincere@april.biz	Romaguera-Crona
1	2	Ervin Howell	Shanna@melissa.tv	Deckow-Crist
2	3	Clementine Bauch	Nathan@yesenia.net	Romaguera-Jacobson
3	4	Patricia Lebsack	Julianne.OConner@kory.org	Robel-Corkery
4	5	Chelsey Dietrich	Lucio_Hettinger@annie.ca	Keebler LLC

--- CSV Data ---

	id	city	country
0	1	New York	USA
1	2	London	UK
2	3	Paris	France
3	4	Tokyo	Japan
4	5	Berlin	Germany
5	6	Delhi	India
6	7	Sydney	Australia
7	8	Moscow	Russia
8	9	Cairo	Egypt
9	10	Beijing	China

--- Merged ETL Pipeline Data ---

	id	name	email
0	1	Leanne Graham	Sincere@april.biz
1	2	Ervin Howell	Shanna@melissa.tv
2	3	Clementine Bauch	Nathan@yesenia.net
3	4	Patricia Lebsack	Julianne.OConner@kory.org
4	5	Chelsey Dietrich	Lucio_Hettinger@annie.ca
5	6	Mrs. Dennis Schulist	Karley_Dach@jasper.info
6	7	Kurtis Weissnat	Telly.Hoeger@billy.biz
7	8	Nicholas Runolfsdottir V	Sherwood@rosamond.me
8	9	Glenna Reichert	Chaim_McDermott@dana.io
9	10	Clementina DuBuque	Rey.Padberg@karina.biz

	company	city	country
0	Romaguera-Crona	New York	USA
1	Deckow-Crist	London	UK
2	Romaguera-Jacobson	Paris	France
3	Robel-Corkery	Tokyo	Japan
4	Keebler LLC	Berlin	Germany
5	Considine-Lockman	Delhi	India
6	Johns Group	Sydney	Australia
7	Abernathy Group	Moscow	Russia
8	Yost and Sons	Cairo	Egypt
9	Hoeger LLC	Beijing	China

Data successfully saved to 'cleaned warehouse profiles.csv'

Practical No. 6

Aim: To set up Apache Airflow and create a daily scheduled DAG to execute sequential ETL pipeline tasks.

```
from datetime import datetime, timedelta
import os

# Create the Airflow DAGs directory
dags_folder = "/content/airflow/dags"
os.makedirs(dags_folder, exist_ok=True)

# Create the DAG Python file
dag_code = '''
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.empty import EmptyOperator

# Default arguments for the workflow
default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "email_on_failure": False,
    "email_on_retry": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

# Define the DAG
with DAG(
    dag_id="university_etl_orchestration",
    default_args=default_args,
    description="Daily ETL pipeline orchestration",
```

```

        schedule=timedelta(days=1),
        catchup=False,
    ) as dag:
# Task 1: Start the pipeline
    start_pipeline = EmptyOperator(
        task_id="start_pipeline"
    )
# Task 2: Execute the extraction script
    execute_extraction = BashOperator(
        task_id="run_extraction_script",
        bash_command="echo 'Running data extraction script...'"
    )
# Task 3: Log successful pipeline completion
    pipeline_complete = BashOperator(
        task_id="log_pipeline_success",
        bash_command='echo "ETL Execution completed successfully at $(date)'"
    )
# Define task dependencies
    start_pipeline >> execute_extraction >> pipeline_complete
    ...
# Save the DAG file
dag_file = os.path.join(dags_folder, "data_pipeline_dag.py")
with open(dag_file, "w") as file:
    file.write(dag_code)

# Display the created DAG file
print("Apache Airflow DAG created successfully!")
print("DAG Name: university_etl_orchestration")
print("Schedule: Daily")

```

```
print("DAG File:", dag_file)
print("\nTask Sequence:")
print("start_pipeline")
print("    ↓")
print("run_extraction_script")
print("    ↓")
print("log_pipeline_success")
```

Output:

```
Apache Airflow DAG created successfully!
DAG Name: university_etl_orchestration
Schedule: Daily
DAG File: /content/airflow/dags/data_pipeline_dag.py

Task Sequence:
start_pipeline
    ↓
run_extraction_script
    ↓
log_pipeline_success
```

Practical No. 7

Aim: To design and implement an ETL pipeline that extracts data from CSV and JSON files, cleans and validates the data, removes invalid records, transforms the data, and loads it incrementally into a relational database.

```
import pandas as pd
import json
import sqlite3
import os

# Step 1: Create Sample CSV Files

csv1 = pd.DataFrame({
    "id": [1, 2, 3],
    "name": ["Alice", "Bob", "Charlie"],
    "age": [22, 25, 30],
    "city": ["Mumbai", "Delhi", "Pune"]
})

csv2 = pd.DataFrame({
    "id": [4, 5, 6],
    "name": ["David", "Eva", "Frank"],
    "age": [28, -5, 35],          # -5 is invalid
    "city": ["Nashik", "Mumbai", "Nagpur"]
})

csv1.to_csv("customers1.csv", index=False)
csv2.to_csv("customers2.csv", index=False)

# Step 2: Extract Data from Multiple CSV Files

df1 = pd.read_csv("customers1.csv")
df2 = pd.read_csv("customers2.csv")

# Combine both CSV datasets
csv_data = pd.concat([df1, df2], ignore_index=True)

print("Combined CSV Data:")
print(csv_data)

# Step 3: Clean and Remove Invalid Records

# Remove duplicate records
csv_data.drop_duplicates(inplace=True)

# Remove records with invalid age
```

```

# Remove records with invalid age
clean_data = csv_data[
    (csv_data["age"] >= 18) &
    (csv_data["age"] <= 100)
].copy()
print("\nData After Removing Invalid Records:")
print(clean_data)

# Step 4: Data Transformation
# Convert names to uppercase
clean_data["name"] = clean_data["name"].str.upper()
# Create a new transformed field
clean_data["category"] = clean_data["age"].apply(
    lambda x: "Young" if x < 30 else "Adult"
)
print("\nTransformed Data:")
print(clean_data)

# Step 5: Create JSON Data
json_data = [
    {"id": 7, "name": "Grace", "age": 24, "city": "Thane"},
    {"id": 8, "name": "Henry", "age": 32, "city": "Mumbai"}
]

with open("customers.json", "w") as file:
    json.dump(json_data, file)

# Step 6: Extract and Transform JSON Data
with open("customers.json", "r") as file:
    json_records = json.load(file)

json_df = pd.DataFrame(json_records)

# Transform selected fields
json_df["name"] = json_df["name"].str.upper()
json_df["category"] = json_df["age"].apply(
    lambda x: "Young" if x < 30 else "Adult"
)

print("\nJSON Data After Transformation:")
print(json_df)

# Step 7: Combine CSV and JSON Data
final_data = pd.concat(
    [clean_data, json_df],
    ignore_index=True
)

```

```

)
print("\nFinal ETL Dataset:")
print(final_data)

# Step 8: Data Validation
# Check required fields and valid age values
valid_data = final_data[
    final_data["id"].notna() &
    final_data["name"].notna() &
    final_data["age"].between(18, 100)
].copy()

print("\nValidated Data:")
print(valid_data)
# Step 9: Connect to Relational Database

connection = sqlite3.connect("etl_database.db")

# Step 10: Incremental Loading

# Create target table if it does not exist
connection.execute("""
CREATE TABLE IF NOT EXISTS customers (
    id INTEGER PRIMARY KEY,
    name TEXT,
    age INTEGER,
    city TEXT,
    category TEXT
)
""")

# Find the latest ID already stored in the database
result = connection.execute(
    "SELECT MAX(id) FROM customers"
).fetchone()

last_id = result[0] if result[0] is not None else 0

# Load only new records
new_records = valid_data[
    valid_data["id"] > last_id
]

if not new_records.empty:

```

```

if not new_records.empty:
    new_records.to_sql(
        "customers",
        connection,
        if_exists="append",
        index=False
    )

    print("\nNew records loaded successfully:")
    print(new_records)
else:
    print("\nNo new records to load.")

# Step 11: Verify Data in Database

database_data = pd.read_sql(
    "SELECT * FROM customers",
    connection
)

print("\nFinal Data in Database:")
print(database_data)

connection.close()

print("\nETL Pipeline completed successfully!")

```

Output:

Combined CSV Data:

	id	name	age	city
0	1	Alice	22	Mumbai
1	2	Bob	25	Delhi
2	3	Charlie	30	Pune
3	4	David	28	Nashik
4	5	Eva	-5	Mumbai
5	6	Frank	35	Nagpur

Data After Removing Invalid Records:

	id	name	age	city
0	1	Alice	22	Mumbai
1	2	Bob	25	Delhi
2	3	Charlie	30	Pune
3	4	David	28	Nashik
5	6	Frank	35	Nagpur

Transformed Data:

	id	name	age	city	category
0	1	ALICE	22	Mumbai	Young
1	2	BOB	25	Delhi	Young
2	3	CHARLIE	30	Pune	Adult
3	4	DAVID	28	Nashik	Young
5	6	FRANK	35	Nagpur	Adult

JSON Data After Transformation:

	id	name	age	city	category
0	7	GRACE	24	Thane	Young
1	8	HENRY	32	Mumbai	Adult

Final ETL Dataset:

	id	name	age	city	category
0	1	ALICE	22	Mumbai	Young
1	2	BOB	25	Delhi	Young
2	3	CHARLIE	30	Pune	Adult
3	4	DAVID	28	Nashik	Young
4	6	FRANK	35	Nagpur	Adult
5	7	GRACE	24	Thane	Young
6	8	HENRY	32	Mumbai	Adult

Validated Data:

	id	name	age	city	category
0	1	ALICE	22	Mumbai	Young
1	2	BOB	25	Delhi	Young
2	3	CHARLIE	30	Pune	Adult
3	4	DAVID	28	Nashik	Young
4	6	FRANK	35	Nagpur	Adult
5	7	GRACE	24	Thane	Young
6	8	HENRY	32	Mumbai	Adult

New records loaded successfully:

	id	name	age	city	category
0	1	ALICE	22	Mumbai	Young
1	2	BOB	25	Delhi	Young
2	3	CHARLIE	30	Pune	Adult
3	4	DAVID	28	Nashik	Young
4	6	FRANK	35	Nagpur	Adult
5	7	GRACE	24	Thane	Young
6	8	HENRY	32	Mumbai	Adult

Final Data in Database:

	id	name	age	city	category
0	1	ALICE	22	Mumbai	Young
1	2	BOB	25	Delhi	Young
2	3	CHARLIE	30	Pune	Adult
3	4	DAVID	28	Nashik	Young
4	6	FRANK	35	Nagpur	Adult
5	7	GRACE	24	Thane	Young
6	8	HENRY	32	Mumbai	Adult

ETL Pipeline completed successfully!

Practical No.8

Aim: To perform data processing using PySpark DataFrames, including schema display, filtering, grouping, aggregation, duplicate removal, joining datasets, and calculating average sales by product category.

```
# Import required libraries
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, sum, count

# Step 1: Create Spark Session

spark = SparkSession.builder \
    .appName("PySpark Practical 8") \
    .getOrCreate()

# Step 2: Create Sample Sales CSV Dataset

sales_data = [
    (1, "Laptop", "Electronics", 50000),
    (2, "Mobile", "Electronics", 20000),
    (3, "Chair", "Furniture", 5000),
    (4, "Table", "Furniture", 8000),
    (5, "Laptop", "Electronics", 50000),
    (6, "Headphones", "Electronics", 3000),
    (7, "Chair", "Furniture", 5000)
]

sales_columns = ["id", "product", "category", "sales"]

sales_df = spark.createDataFrame(
    sales_data,
    sales_columns
)

# Save dataset as CSV
sales_df.write.mode("overwrite").option(
    "header", True
).csv("sales_data.csv")

# Step 3: Read CSV Dataset

df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv("sales_data.csv")
```

```

.csv("sales_data.csv")

print("Sales Dataset:")
df.show()

# Step 4: Display Schema

print("Dataset Schema:")
df.printSchema()

# Step 5: Filtering
# Display products with sales greater than 10000

print("Filtered Data:")
df.filter(df.sales > 10000).show()

# Step 6: Grouping and Aggregation
# Calculate total sales by category
print("Total Sales by Category:")

df.groupBy("category") \
    .agg(sum("sales").alias("total_sales")) \
    .show()

# Step 7: Remove Duplicate Records

print("Data After Removing Duplicates:")

unique_df = df.dropDuplicates()

unique_df.show()

# Step 8: Create Second Dataset for Join
product_data = [
    ("Laptop", "Dell"),
    ("Mobile", "Samsung"),
    ("Chair", "IKEA"),
    ("Table", "IKEA"),
    ("Headphones", "Sony")
]

product_columns = ["product", "brand"]

product_df = spark.createDataFrame(
    product_data,
    product_columns

```

```

product_data,
    product_columns
)

print("Product Details:")
product_df.show()

# Step 9: Join Two Datasets

print("Joined Dataset:")

joined_df = unique_df.join(
    product_df,
    on="product",
    how="inner"
)

joined_df.show()

# Step 10: Calculate Average Sales by Product Category
print("Average Sales by Product Category:")

df.groupBy("category") \
    .agg(avg("sales").alias("average_sales")) \
    .show()

# Step 11: Stop Spark Session

spark.stop()

print("PySpark practical completed successfully!")

```

Output:

Sales Dataset:

id	product	category	sales
4	Table	Furniture	8000
5	Laptop	Electronics	50000
6	Headphones	Electronics	3000
7	Chair	Furniture	5000
1	Laptop	Electronics	50000
2	Mobile	Electronics	20000
3	Chair	Furniture	5000

Dataset Schema:

```

root
 |-- id: integer (nullable = true)
 |-- product: string (nullable = true)
 |-- category: string (nullable = true)
 |-- sales: integer (nullable = true)

```

Filtered Data:

id	product	category	sales
5	Laptop	Electronics	50000
1	Laptop	Electronics	50000
2	Mobile	Electronics	20000

Filtered Data:

```
+---+-----+-----+-----+
| id|product|  category|sales|
+---+-----+-----+-----+
|  5| Laptop|Electronics|50000|
|  1| Laptop|Electronics|50000|
|  2| Mobile|Electronics|20000|
+---+-----+-----+-----+
```

Total Sales by Category:

```
+-----+-----+
| category|total_sales|
+-----+-----+
|Electronics|    123000|
| Furniture|     18000|
+-----+-----+
```

Data After Removing Duplicates:

```
+---+-----+-----+-----+
| id|  product|  category|sales|
+---+-----+-----+-----+
|  4|   Table| Furniture| 8000|
|  6|Headphones|Electronics| 3000|
|  7|   Chair| Furniture| 5000|
|  5|   Laptop|Electronics|50000|
|  2|   Mobile|Electronics|20000|
|  3|   Chair| Furniture| 5000|
|  1|   Laptop|Electronics|50000|
+---+-----+-----+-----+
```

Product Details:

```
+-----+-----+
| product| brand|
+-----+-----+
| Laptop| Dell|
| Mobile| Samsung|
| Chair| IKEA|
| Table| IKEA|
|Headphones| Sony|
+-----+-----+
```

Joined Dataset:

```
+---+-----+-----+-----+-----+
| product| id|  category|sales| brand|
+---+-----+-----+-----+-----+
| Laptop| 1|Electronics|50000| Dell|
| Laptop| 5|Electronics|50000| Dell|
| Mobile| 2|Electronics|20000| Samsung|
| Chair| 3| Furniture| 5000| IKEA|
| Chair| 7| Furniture| 5000| IKEA|
| Table| 4| Furniture| 8000| IKEA|
|Headphones| 6|Electronics| 3000| Sony|
+---+-----+-----+-----+-----+
```

Average Sales by Product Category:

```
+-----+-----+
| category|average_sales|
+-----+-----+
|Electronics|    30750.0|
| Furniture|     6000.0|
+-----+-----+
```

PySpark practical completed successfully!

Practical No. 9

Aim: To build an end-to-end ETL pipeline for an e-commerce dataset that extracts CSV data, performs data cleaning and transformation, loads it into a SQL database, generates a report, and handles both historical and newly arriving data.

```
import pandas as pd
import sqlite3

# Step 1: Create E-Commerce CSV Files

customers = pd.DataFrame({
    "customer_id": [1, 2, 3, 4],
    "customer_name": ["Alice", "Bob", "Charlie", "David"],
    "city": ["Mumbai", "Delhi", "Pune", "Nashik"]
})

products = pd.DataFrame({
    "product_id": [101, 102, 103, 104],
    "product_name": ["Laptop", "Mobile", "Headphones", "Keyboard"],
    "category": ["Electronics", "Electronics", "Accessories",
"Accessories"],
    "price": [50000, 20000, 3000, 1500]
})

orders = pd.DataFrame({
    "order_id": [1001, 1002, 1003, 1004],
    "customer_id": [1, 2, 3, 4],
    "product_id": [101, 102, 103, 104],
    "quantity": [1, 2, 2, 3]
})

payments = pd.DataFrame({
    "payment_id": [501, 502, 503, 504],
    "order_id": [1001, 1002, 1003, 1004],
    "payment_method": ["UPI", "Card", "UPI", "Cash"],
    "payment_status": ["Completed", "Completed", "Completed", "Pending"]
})

# Save datasets as CSV files
customers.to_csv("customers.csv", index=False)
products.to_csv("products.csv", index=False)
orders.to_csv("orders.csv", index=False)
payments.to_csv("payments.csv", index=False)
```

```

# Step 2: Extract Data from CSV Files

customers_df = pd.read_csv("customers.csv")
products_df = pd.read_csv("products.csv")
orders_df = pd.read_csv("orders.csv")
payments_df = pd.read_csv("payments.csv")

print("CSV data extracted successfully.")

# Step 3: Data Cleaning

# Remove duplicate records
customers_df.drop_duplicates(inplace=True)
products_df.drop_duplicates(inplace=True)
orders_df.drop_duplicates(inplace=True)
payments_df.drop_duplicates(inplace=True)

# Remove invalid product prices
products_df = products_df[products_df["price"] > 0]

# Remove invalid order quantities
orders_df = orders_df[orders_df["quantity"] > 0]

print("\nData cleaning completed.")

# Step 4: Transform and Combine Data

# Join orders with customer information
report_df = orders_df.merge(
    customers_df,
    on="customer_id",
    how="left"
)

# Join product information
report_df = report_df.merge(
    products_df,
    on="product_id",
    how="left"
)

# Join payment information
report_df = report_df.merge(
    payments_df,
    on="order_id",

```

```

payments_df,
    on="order_id",
    how="left"
)

# Calculate total order amount
report_df["total_amount"] = (
    report_df["quantity"] * report_df["price"]
)

print("\nTransformed E-Commerce Data:")
print(report_df)

# Step 5: Load Data into SQL Database
connection = sqlite3.connect("ecommerce.db")

# Store the final data in SQL table
report_df.to_sql(
    "ecommerce_orders",
    connection,
    if_exists="replace",
    index=False
)

print("\nData loaded into SQL database successfully.")

# Step 6: Generate Report Using SQL
report = pd.read_sql_query("""
    SELECT
        category,
        COUNT(order_id) AS total_orders,
        SUM(quantity) AS total_quantity,
        SUM(total_amount) AS total_sales
    FROM ecommerce_orders
    GROUP BY category
""", connection)

print("\n--- E-Commerce Sales Report ---")
print(report)

# Step 7: Handle Newly Arriving Data

new_orders = pd.DataFrame({
    "order_id": [1005],
    "customer_id": [1],
    "product id": [102],

```

```

"customer_id": [1],
  "product_id": [102],
  "quantity": [1]
})

# Add new order to the existing orders
orders_df = pd.concat(
    [orders_df, new_orders],
    ignore_index=True
)

# Process the new order
new_order_report = new_orders.merge(
    customers_df,
    on="customer_id",
    how="left"
)

new_order_report = new_order_report.merge(
    products_df,
    on="product_id",
    how="left"
)

new_order_report["total_amount"] = (
    new_order_report["quantity"] *
    new_order_report["price"]
)

# Add default payment information
new_order_report["payment_id"] = 505
new_order_report["payment_method"] = "UPI"
new_order_report["payment_status"] = "Completed"

# Select same columns as database table
new_order_report = new_order_report[
    report_df.columns
]

# Insert newly arriving record into SQL database
new_order_report.to_sql(
    "ecommerce_orders",
    connection,
    if_exists="append",
    index=False
)

```

```

index=False
)

print("\nNewly arriving data loaded successfully.")

# Step 8: Final Database Report

final_report = pd.read_sql_query("""
    SELECT
        category,
        COUNT(order_id) AS total_orders,
        SUM(quantity) AS total_quantity,
        SUM(total_amount) AS total_sales
    FROM ecommerce_orders
    GROUP BY category
""", connection)

print("\n--- Final Report After New Data ---")
print(final_report)

# Close Database Connection

connection.close()

print("\nEnd-to-end ETL pipeline completed successfully!")

```

Output:

CSV data extracted successfully.

- Data cleaning completed.

Transformed E-Commerce Data:

	order_id	customer_id	product_id	quantity	customer_name	city	\
0	1001	1	101	1	Alice	Mumbai	
1	1002	2	102	2	Bob	Delhi	
2	1003	3	103	2	Charlie	Pune	
3	1004	4	104	3	David	Nashik	

	product_name	category	price	payment_id	payment_method	payment_status	\
0	Laptop	Electronics	50000	501	UPI	Completed	
1	Mobile	Electronics	20000	502	Card	Completed	
2	Headphones	Accessories	3000	503	UPI	Completed	
3	Keyboard	Accessories	1500	504	Cash	Pending	

	total_amount
0	50000
1	40000
2	6000
3	4500

Data loaded into SQL database successfully.

--- E-Commerce Sales Report ---

	category	total_orders	total_quantity	total_sales
0	Accessories	2	5	10500
1	Electronics	2	3	90000

Newly arriving data loaded successfully.

--- Final Report After New Data ---

	category	total_orders	total_quantity	total_sales
0	Accessories	2	5	10500
1	Electronics	3	4	110000

End-to-end ETL pipeline completed successfully!

Practical No. 10

Aim: To design and implement an end-to-end data engineering solution involving data ingestion, cleaning, transformation, storage, data warehouse design, and reporting through an interactive dashboard.

